



WhatsApp Catalog Engine

Complete Technical Architecture, System Roadmap, Stack Specifications, and Verification Guardrails Optimized for Zero-Cost Deployments

Part I: System Goals

Establishing target technical parameters to deliver instant loading speeds over local MTN/Airtel connections without running server costs

Core Business & Technical Goals

Business Objectives

Equip small business boutiques in Benin City with zero-barrier digital storefront profiles, eliminating the need for slow, manual media exchanges.

Onboard fashion and product vendors with instant value, converting free trial accounts into paid ₦3,000 monthly subscriptions.

Technical Objectives

Implement a unified multi-tenant architecture utilizing path routing variables to serve hundreds of unique vendors from a single active server.

Minimize client-side network payloads to keep processing fast, loading pages in milliseconds over unstable mobile network signals.

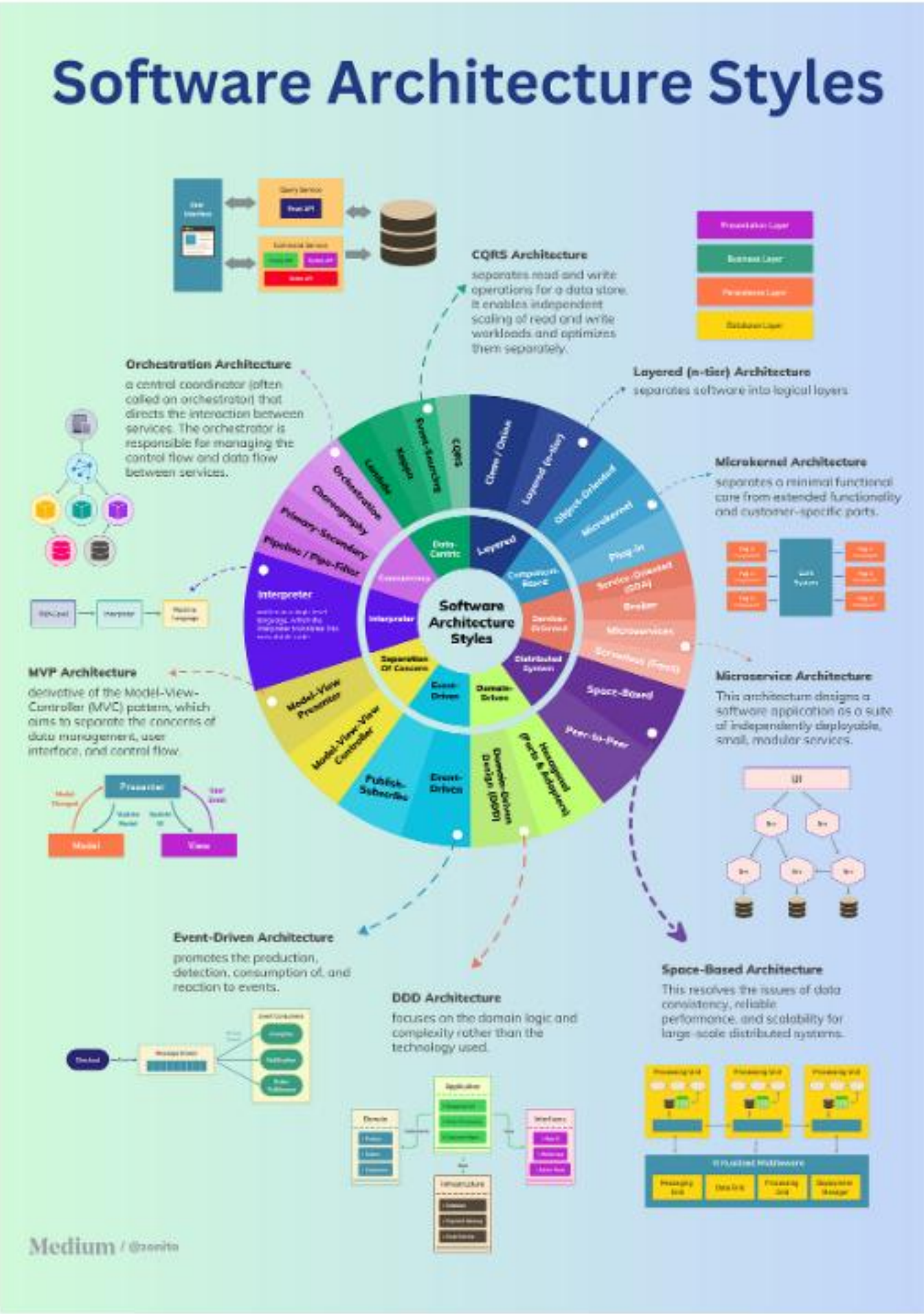
Zero-Cost System Architecture

Serverless Network Routing

By utilizing on-demand serverless hosting functions, the engine remains dormant and consumes nothing until a customer hits a vendor's slug.

Database responses return structured product listings within milliseconds, bypassing slow startup delays common on standard free server tiers.

Centralized assets pull from an optimized global network layer, maximizing display efficiency on light mobile phone screens.



The Native Serverless Stack



Next.js Framework

Processes directory paths and compiles dynamic product data into light-weight static HTML before transmitting to customer screens.



Supabase Engine

Provides a serverless relational Postgres core with built-in asset storage, comfortably serving up to 500MB of vendor files for free.



Vercel Hosting

Distributes public boutique directories through global data centers, eliminating page wake-up lag and ongoing hosting expenses.

Part II: Database & UI

Mapping out standard table schemas, developmental phases, and UI requirements to secure clean execution

Phase-Based Development Roadmap

1. Database

Establish PostgreSQL tables, foreign key constraints, and performance indexes inside Supabase dashboard.

2. Dynamic Routes

Deploy next-router variable mappings, pulling layout data according to incoming domain path slugs.

3. Asset Pipeline

Incorporate client-side canvas compression routines to scale raw uploaded photos down to under 150KB.

4. Push Checkout

Code the client state cart handler and configure the message compiler string for instant WhatsApp redirects.

Database Schema Setup

Table Name	Primary Columns & Types	Index Strategy	Cost Impact
stores	id (UUID), slug (VARCHAR), whatsapp_number (VARCHAR)	Unique Index on slug	≈0 (Negligible bytes)
products	id (UUID), store_id (UUID), name (VARCHAR), price (NUMERIC)	B-tree Index on store_id	≈0 (Text data records)
storage.objects	id (UUID), bucket_id (VARCHAR), owner (UUID)	Internal Postgres Index	≈0 (Max 150KB per item)

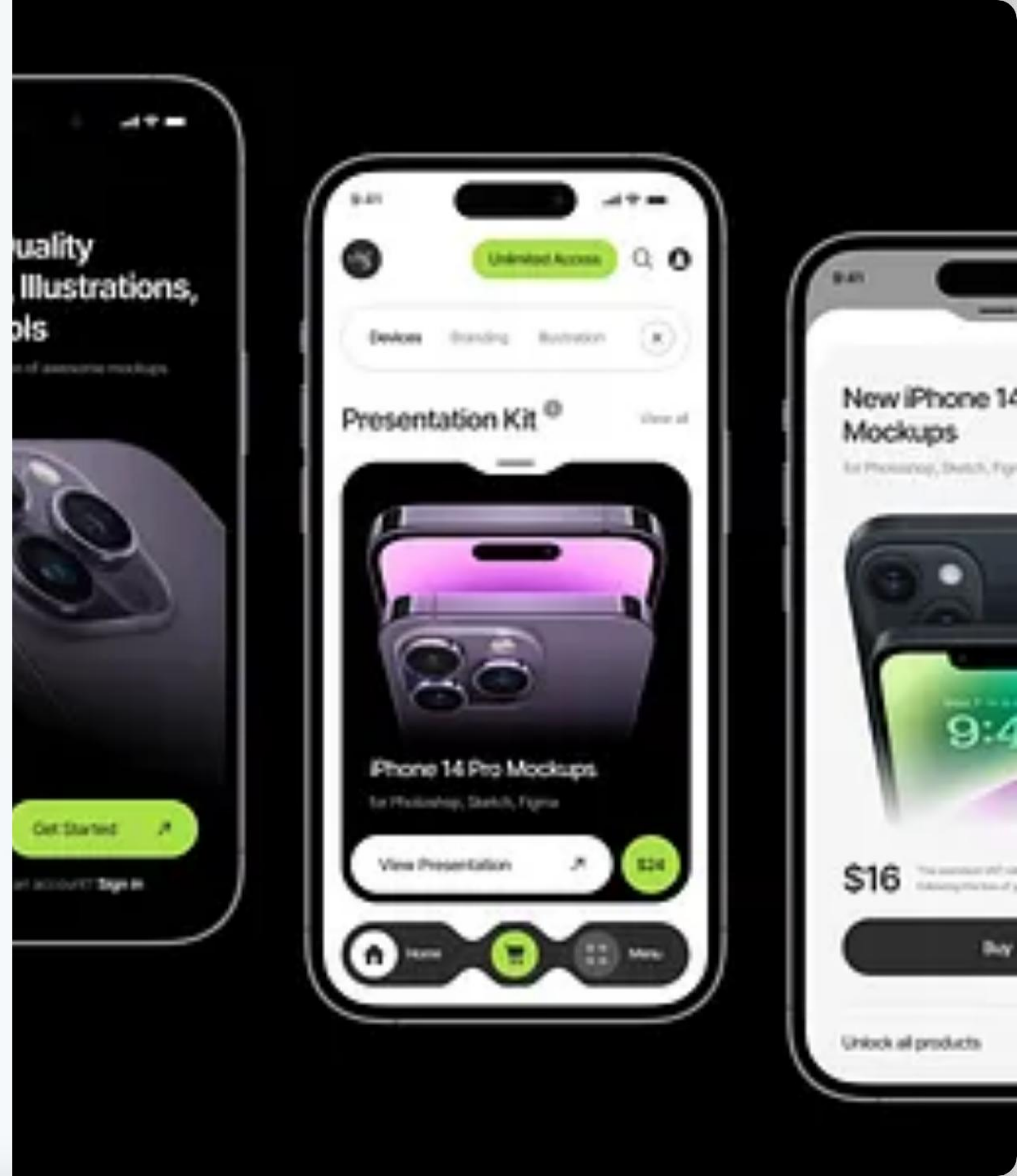
Ultra-Lightweight UX Engine

Dynamic CSS Styling

The storefront eliminates heavy external frame libraries. All presentation elements render using native modular utility styles.

By stripping out rendering weight, client browsers compile layout pages within micro-seconds of receiving the initial package.

This layout operates without external script downloads, meaning even old mobile phones display catalogs without slowing down.



Key System Performance Metrics

NO

Lifetime Server Maintenance

Engine Operational Targets

By leveraging serverless logic and standard PostgreSQL constraints on the Supabase free tier, you bypass constant server leasing payments.

The target initial page weight remains under ****100KB****, ensuring files execute seamlessly even with restricted carrier connectivity.

This keeps maintenance costs at absolute zero while providing enterprise-grade processing capabilities.

Development Phase Guardrails

- ✓ **Implement Client-Side Canvas Image Compression**
Raw mobile phone images can exceed 4MB, which would quickly exhaust Supabase free storage buckets. Automatically scale and compress images on the client side before triggering uploads.
- ✓ **De-couple Financial Intermediation**
Do not build automated bank card processors into early stages. Direct checkout payloads to WhatsApp as secure bank transfer requests, bypassing expensive dynamic integration overhead.
- ✓ **Employ Local Storage Cart Management**
Retain selected checkout profiles inside standard client-side browser memory instead of pushing draft tables to database servers. This keeps API request loops minimal and lightning-fast.

Engineering Launch Checklist

Verify storage limits, test routing parameters locally, and deploy dynamic boutique links.

Write clean code. Scale with ~~NO~~ hosting. Monetize local markets.

Image Sources



https://miro.medium.com/v2/resize:fit:1200/1*46JX-o6xxMaMmOmTLwyLYQ.png

Source: medium.com



<https://cdn.dribbble.com/userupload/3508164/file/original-3cfe5ff929e02552fee29f0cc52ca05b.png?format=webp&resize=400x300&vertical=center>

Source: dribbble.com